Manager. Specify the name, target, size and click on *Start*. The emulator will be launched.

Now click on *MyPhonegapDemo Project* and then right click *Run As>Android Application*. Running the project in the emulator will result in the screen shot shown in Figure 3.

Congratulations. You are now done creating a simple application.

Exploring PhoneGap APIs

As I have already given you the list of APIs included in PhoneGap, let's explore them to see what benefit can be obtained from each and how they can be included in your application to make it feature rich. These APIs give you the advantage of accessing native device features like the accelerometer, the compass, contacts, etc. Let's start playing around with these APIs.

Accelerometer: This is a motion sensor that can detect a change in movement with respect to the device's orientation in all the three axes. It allows you to change the position of an object in your application with every move of your device.

Usage: This can be implemented in racing games.

The methods for using the accelerometer are:

1. *getCurrentAcceleration()*—Used to get the current acceleration along three axes x,y,z.
2. *watchAcceleration()* - Used to get the acceleration with some regular time interval.
3. *clearWatch()* - Used to stop the watch that you started with *watchAcceleration()*

The arguments to pass through these methods are:

1. *accelerometerSuccess()* - This method will be called when we get successful acceleration from the device.
2. *accelerometerError()* - This method will be called when we fail to receive acceleration from the device.
3. *accelerometerOptions* – This will let you set the frequency for the time interval used in *watchAcceleration()*

So let's try to put these things up in the code to see it in action.

Replace *index.html* with *accelerometer.html*

Note: The code below is for the *watchAcceleration* method.

```
<html>  <head>
    <title>Accelerometer API</title>
    <script type="text/javascript" charset="utf-8"
src="cordova-2.7.0.js"></script>
    <script>
    var watch=null;              // variable to be used inside
watchAcceleration method
    document.addEventListener("deviceready", deviceReady,
false);   (1)

function deviceReady() {    (2)
    startWatch();  (3)
}

function startWatch() {
    var options = { frequency: 5000 };    (4)
    watch = navigator.accelerometer.
watchAcceleration(onSuccess, onError, options);  (5)
}
function onSuccess(acceleration) {
    var anyElement = document.getElementById('acceleration');
(6)
    anyElement.innerHTML = 'Acceleration X: ' +
acceleration.x + '<br />' +
                        'Acceleration Y: ' + acceleration.y +
'<br />' +
                        'Acceleration Z: ' + acceleration.z +
'<br />' +
                        'Timestamp: '      + acceleration.
timestamp + '<br />';     (7)
}
function onError() {  (8)
    alert('onError!');
}
function stopWatch() {
    if (watch) {   (9)
        navigator.accelerometer.clearWatch(watch);   (10)
        watch= null;
    }
}
    </script> </head> <body>
  <p id="acceleration">Waiting for acceleration</p>
  </body> </html>
```

The output of the above code is shown in Figure 4.

Explanation: The code above shows the acceleration with an increasing timestamp.

Reference 1 in the code shows the *document.addEventListener* function, which depicts the wait for PhoneGap to load. References 2 and 3 depict that when PhoneGap is loaded, the deviceReady method will call *startwatch()*, i.e., the user defined method. Reference 4 shows that inside the *startwatch()*, there is a variable called the options ID, depicting the frequency for the timestamp. Reference 5 shows that the method *watchAcceleration()* fetches the current acceleration along the x,y and z axes. The arguments passed will be called at the time of success or failure. References 6 and 7 indicate that in case of success, the body element 'Waiting for acceleration' will be replaced by 'acceleration' with respect to x, y and z. Reference 8 shows that in case of an error, an alert will be shown with an error message. References 9 and 10 show that in the *stopWatch()* method, if the watch is not null, it means the *watchAcceleration()* method has been called before; so use *clearwatch()* to stop watching the accelerometer.